

Resumen Exhaustivo – Redes y Comunicaciones

Basado en: Chapter_2_v8.0 (Aplicación), CCRI sockets, y (Lossless – Lossy).

Objetivo: estudiar para el control (mañana 7:00 pm) con foco en comprensión y retención.

1) Capa de Aplicación (Kurose & Ross, Chapter_2_v8.0)

1.1 Panorama y metas

- Principios de aplicaciones de red: modelos de servicio de transporte (TCP/UDP), arquitecturas (cliente-servidor, P2P).
- Estudio de protocolos de aplicación populares: HTTP, SMTP/IMAP, DNS.
- Programación de aplicaciones de red con la API de sockets (UDP/TCP).

1.2 Arquitecturas de aplicación

- Cliente–Servidor: servidor siempre encendido (IP fija, data centers), clientes con IP dinámica y conexiones intermitentes. Ejemplos: HTTP, IMAP, FTP.
- P2P: sin servidor persistente; los pares actúan como clientes y servidores (auto-escalable pero de gestión compleja). Ejemplos: BitTorrent, VoIP (históricamente), streaming P2P.

1.3 Procesos y Sockets

- Proceso: programa en ejecución dentro de un host. Entre hosts, los procesos se comunican intercambiando mensajes.
- Cliente: inicia comunicación. Servidor: espera ser contactado.
- Socket: interfaz entre aplicación y transporte; la app 'empuja' mensajes a través del socket.
- Dirección de proceso: (IP, puerto). Ejemplos: HTTP 80, SMTP 25.

1.4 Requerimientos de servicio de transporte

- Integridad de datos: algunas apps requieren 100% confiabilidad (transferencia de archivos); otras toleran pérdidas (audio).
- Timing (retardo): voz en tiempo real y juegos requieren baja latencia.
- Throughput: multimedia requiere mínimo de tasa; apps elásticas se adaptan.
- Seguridad: cifrado, integridad, autenticación (TLS sobre TCP).

1.5 Servicios de transporte en Internet

- TCP: confiable, en orden, control de flujo, control de congestión, orientado a conexión. No garantiza timing ni throughput mínimo. TLS puede proveer cifrado.
- UDP: no confiable, sin control de flujo/congestión, sin conexión; elegido por simplicidad y baja latencia cuando pérdidas son tolerables.
- Asignación típica: HTTP/SMTP/IMAP → TCP; DNS, streaming/juegos → UDP (dependiendo del caso).

1.6 Web y HTTP

- Página web: objeto base HTML + objetos referenciados (imágenes, etc.), cada uno con URL.
- HTTP usa TCP (puerto 80). Protocolo sin estado (stateless).
- HTTP no persistente: 1 objeto por conexión; demora ≈ 2 RTT + tiempo de transmisión por objeto.
- HTTP persistente (HTTP/1.1): múltiples objetos en una conexión; reduce RTTs y overhead.
- Mensajes: Request (métodos GET/POST/HEAD/PUT), Response (códigos: 200 OK, 301, 400, 404, 505).
- Cookies: mecanismo para mantener estado entre cliente y servidor (cabeceras Set-Cookie / Cookie + almacenamiento local).
- Cachés web (proxies): reducen latencia y tráfico; Conditional GET (If-Modified-Since) y 304 Not Modified.
- HTTP/2: priorización y división en frames para mitigar head-of-line (HOL) y permitir interleaving; server push.
- HTTP/3: sobre QUIC (UDP), seguridad integrada, control por flujo/errores por objeto; reduce estancamiento por pérdidas.

1.7 Correo electrónico: SMTP, IMAP

- Componentes: agentes de usuario (UA), servidores de correo, protocolo SMTP.
- SMTP: usa TCP (25), persistente; fases: saludo (HELO/EHLO), transferencia (MAIL FROM, RCPT TO, DATA), cierre. Mensajes en 7-bit ASCII; terminación CRLF.CRLF.
- Formato mensaje (RFC 822/5322): cabeceras (To, From, Subject), cuerpo (ASCII; adjuntos vía MIME y codificaciones como Base64).
- Acceso al correo: IMAP/POP/HTTP (webmail). IMAP mantiene mensajes en servidor con carpetas, POP descarga.

1.8 DNS: Sistema de Nombres de Dominio

- Servicio: traducción nombre $\leftrightarrow$ IP, alias de hosts y mail (MX), balanceo por réplica (múltiples IPs por nombre).
- Arquitectura jerárquica y distribuida: Root $\rightarrow$ TLD (.com, .pe, .edu) $\rightarrow$ Autoritativos; servidores locales con caché.
- Resolución iterativa vs recursiva; caché con TTL y posibles desactualizaciones.
- Registros (RR): A (nombre $\rightarrow$ IPv4), AAAA (IPv6), NS (nameserver), CNAME (alias), MX (mail exchanger).
- Mensajes DNS: cabecera (id, flags), secciones de preguntas, respuestas, autoridad y adicional.
- Seguridad: ataques DDoS, envenenamiento de caché; DNSSEC proporciona autenticación e integridad.

1.9 P2P y BitTorrent

- Distribución de archivos: cliente-servidor vs P2P; tiempo de distribución client-server crece linealmente con N, P2P aprovecha subida agregada de los pares.
- BitTorrent: archivo dividido en 'chunks'; tracker lista pares; política 'rarest first'; 'tit-for-tat' (subo a quienes más me suben); 'optimistic unchoke'.

1.10 Streaming de video y CDNs

- Vídeo digital: frames por segundo; compresión por redundancia espacial (dentro del frame) y temporal (entre frames).
- CBR vs VBR; ejemplos: MPEG-1/2/4.
- Desafíos: jitter de red, pérdidas; búfer de reproducción y retardo de playout.

- DASH (Dynamic Adaptive Streaming over HTTP): el cliente elige tasa/codificación por chunk según ancho de banda; manifiesto (MPD) con URLs de calidades.
- CDNs: copias distribuidas (deep/in access networks vs clusters en POPs). Resolución vía CNAMEs y DNS para dirigir al nodo 'cercano'.

1.11 Programación con Sockets (vista general)

- UDP: sin conexión; se adjunta destino en cada datagrama; posible desorden/pérdida.
- TCP: conexión; flujo de bytes fiable y en orden.
- API básica (Python/C): crear socket, enviar/recibir, cerrar.

Resumen Cap. 2

Temas transversales: centralizado vs descentralizado; stateless vs stateful; escalabilidad; confiabilidad vs latencia; complejidad en el borde (end systems).

2) Programación de Sockets en C (CCRI sockets)

2.1 Conceptos básicos

- Socket: interfaz entre aplicación y red. La app envía/recibe datos a través de un socket.
- Tipos esenciales: SOCK_STREAM (TCP) – fiable, ordenado, orientado a conexión; SOCK_DGRAM (UDP) – no fiable, sin orden, sin conexión.
- Cada host tiene direcciones IP y 65,536 puertos; algunos bien conocidos (FTP 20/21, Telnet 23, HTTP 80).

2.2 Creación y asociación de sockets

- socket(domain, type, protocol): crea el descriptor. domain: PF_INET (IPv4). type: SOCK_STREAM o SOCK_DGRAM. protocol: usualmente 0.
- bind(sock, &addr, size): asocia IP/puerto local. Para UDP que solo envía, puede omitirse; para recibir, se requiere. En TCP se suele omitir y el sistema asigna un puerto efímero al conectar.
- listen(sock, queuelen): marca el socket como pasivo (TCP) para aceptar conexiones entrantes (no bloquea).
- accept(sock, &name, &namelen): bloqueante; crea un nuevo socket para la conexión con el cliente, dejando el original escuchando.
- connect(sock, &name, namelen): lado activo establece la conexión (bloqueante).

2.3 Envío y recepción de datos

- TCP: send(sock, buf, len, flags) / recv(sock, buf, len, flags) – bloquean hasta que se escriben/leen bytes (a nivel de buffer de socket).
- UDP: sendto(sock, buf, len, flags, &addr, addrlen) / recvfrom(sock, buf, len, flags, &addr, &addrlen) – incluyen dirección/puerto por datagrama.
- close(sock): cierra la conexión (TCP) y libera el puerto.

2.4 Estructuras de dirección y endianness

- struct sockaddr (genérica) y sockaddr_in (IPv4): sin_family=AF_INET, sin_port (16 bits), sin_addr (32 bits), sin_zero (padding).
- Orden de bytes: la red usa 'network byte order' (big-endian). Conversiones: htons/htonl (host→network), ntohs/ntohl (network→host).
- Funciones útiles: gethostname, gethostbyaddr, inet_addr (dotted→long), inet_ntoa (long→dotted).

2.5 Bloqueo y multiplexación (select)

- Muchas llamadas son bloqueantes: accept, connect, recv/recvfrom, send/sendto (hasta buffer).
- Estrategias: multihilo/multiproceso, sockets no bloqueantes, o select().
- select(nfds, &readfds, &writefds, &exceptfds, &timeout): espera hasta que alguno esté listo (lectura/escritura/excepción); puede ser no bloqueante (timeout=0) o con timeout.
- Macros: FD_ZERO, FD_SET, FD_ISSET. Útiles para manejar múltiples sockets en un solo hilo.

2.6 Liberación de puertos y señales

- Salidas abruptas pueden dejar puertos en TIME_WAIT; eventualmente se liberan. Manejadores de señal (SIGINT/SIGTERM) ayudan a cerrar ordenadamente.
- Buenas prácticas: cerrar sockets, verificar errores de llamadas de sistema.

3) Compresión: Lossless y Lossy (Lossless – Lossy)

3.1 Introducción y categorías

- Compresión: eliminar y luego (posiblemente) reintroducir redundancia; puede eliminar datos 'insignificantes'.
- Dos categorías: Lossless (sin pérdida) y Lossy (con pérdida).
- Analogía: jugo concentrado (se quita 'agua'; para lossless se recupera exactamente, para lossy no).

3.2 Lossless (sin pérdida)

- Recuperación exacta de los datos originales. Útil en texto, código fuente, ejecutables.
- Ratios típicos: 2:1 a 8:1 según contenido.
- Algoritmos: Huffman (árboles binarios/prioridades), LZW (Lempel–Ziv–Welch).
- Formatos/Ejemplos: ZIP/PKZIP, PNG (imágenes con compresión sin pérdida), GIF (LZW, 256 colores).

3.3 Lossy (con pérdida)

- Reconstrucción aproximada; optimiza percepción humana (descarta componentes menos perceptibles).
- Usado en señales (voz, música) e imágenes/vídeo.
- Ratios muy altos (decenas:1) posibles, a costa de artefactos.
- Ejemplos: JPEG (fotos), MPEG (vídeo), MP3 (audio), compresión de voz en móviles.

3.4 DCT y Wavelets

- DCT (Transformada Discreta del Coseno): representa datos como suma de cosenos a distintas frecuencias. Base de JPEG (imágenes) y MP3 (audio).
- 2D-DCT para imágenes (raster) → paso a dominio de frecuencias.
- Wavelets: funciones de soporte finito, mejores para bordes/curvas; útiles en compresión de imágenes con contornos (e.g., JPEG2000).

3.5 JPEG (flujo de trabajo)

- Dividir imagen en bloques 8×8 píxeles.
- Aplicar 2D-DCT a cada bloque para obtener coeficientes de frecuencia.
- Cuantizar: más gruesa para frecuencias altas (visión humana menos sensible).
- Recorrida zig-zag para agrupar coeficientes; compresión entropía (Huffman).
- Efectos de compresión: desenfoque/'ringing', bloques visibles a altas compresiones.

3.6 Formatos de imagen en la Web

- GIF 89a: paleta de hasta 256 colores, LZW (lossless), transparencia binaria y animación; hoy superado por PNG en estáticos.
- PNG: lossless, soporta transparencia alfa (8 bits). Preferido para logos, gráficos con bordes nítidos, texto.
- JPEG: diseñado para fotografías; siempre con pérdida (perception-based). No apto para texto/bordes nítidos.
- SVG: formato vectorial (XML); ideal para gráficos escalables, no para fotografías.

- JPEG 2000: wavelets y transmisión progresiva; adopción web limitada.

3.7 Codecs y contenedores; streaming vs descarga

- Codec: algoritmo de compresión (audio/vídeo). Contenedor: formato de archivo que empaqueta pistas (vídeo, audio, subtítulos) y metadatos.
- Imágenes: no se distingue contenedor/codec (p.ej., PNG contiene datos PNG).
- Descarga: recurso completo se almacena; Streaming: reproducción sin descarga completa (e.g., live cam).
- Consideraciones de DRM/redistribución y formatos propietarios para streaming.

3.8 SMTP y Base64 (limitaciones y solución)

- SMTP clásico: 7-bit ASCII; sin autenticación ni cifrado por defecto; proclive a abuso (spam).
- Base64: convierte binarios en texto ASCII (bloques 24 bits $\rightarrow$ 4 $\times$ 6 bits $\rightarrow$ 25% overhead). Usado en MIME para adjuntos en correo.

3.9 Comparativas y elección de formato

- Fotografías: JPEG (lossy) – tamaño pequeño, artefactos a alta compresión.
- Gráficos/Logos: PNG (lossless) o SVG (vector).
- Animaciones simples: GIF (o APNG donde se soporte).
- Bordes/curvas complejas: Wavelets/JPEG2000 (si hay soporte).

Fin del resumen. Recomendación: usar este documento junto con mapas mentales y flashcards para reforzar puertos, métodos y diferencias clave.